HIPSTER

We strongly believe in assessing talent based on real-world problem-solving skills and practical abilities rather than solely focusing on experience, degrees, or certifications. Here's why:

Why Practical Assessments Matter
In today's rapidly evolving AI and ML landscape, the ability to implement, adapt, and deliver innovative solutions is critical. While traditional qualifications highlight foundational knowledge, real-world assessments:

1. **Demonstrate Hands-On Expertise**: They show your ability to apply concepts in practical scenarios.
2. **Show Problem-Solving Skills**: They allow us to see how you approach challenges, navigate complexities, and create solutions.
3. **Focus on Deliverables**: By evaluating actual outcomes, we emphasize results and impact over credentials.
4. **Foster Growth**: This process helps identify individuals who are not just skilled but also ready to learn and grow with us.

Your Task
We have designed three tasks, each reflecting real-world challenges you might encounter in this role. Out of the three tasks listed, **please select the one** where you feel most confident and align with your skills.

Once you've selected your task, you can begin working on it. After completing it, please respond to this email with your readiness.

What Happens Next
We will schedule a video meeting where you will:

1. **Showcase Your Work**: Demonstrate your setup, approach, and solution.
2. **Discuss Challenges**: Share your thought process, any obstacles faced, and how you resolved them.
3. **Receive Feedback**: We will evaluate your work based on technical skills, problem-solving, and scalability of your solution.

Why This Process is Beneficial
This process is designed not only to help us assess your skills but also to give you a platform to shine through your work. It reflects the kind of projects you'll work on here and ensures a mutual alignment of expectations.

HIPSTER

Task 1: Translation Model Fine-tuning and Deployment

Prompt:
Translation models often fail to account for cultural nuances, idioms, and community-specific expressions. For example, translating रस्सी जल गयी, बल नहीं गया using a generic model like Google Translate may yield incorrect results. The accurate translation is *Even after one hits rock bottom, their arrogance remains unchanged.*

Your task is to fine-tune a large translation model to better handle such cultural idioms and nuances.

Requirements:

1. **Model Selection**: Choose a pre-trained translation model (e.g., MarianMT, T5, M2M-100).
2. **Dataset**: Use a small, labelled dataset with examples of culturally nuanced idioms and their translations. Examples can be sourced from:
    - Tatobbaidioms Dataset
    - OpenSubtitles Dataset
3. **Tasks**:
    - Fine-tune the translation model on this dataset using frameworks like Hugging Face Transformers.
    - Deploy the fine-tuned model on a cloud platform (AWS, Azure, or GCP).
    - Build a REST API using frameworks like FastAPI or Flask to expose the model for predictions.
4. **Evaluation**:
    - Use BLEU or METEOR as evaluation metrics to measure translation accuracy.
    - Demonstrate performance improvements by comparing pre- and post-fine-tuning metrics.

Evaluation Criteria:

- **Technical Skills**: Leveraging frameworks like Hugging Face, LangChain, and deploying on cloud platforms.
- **Problem-Solving**: Handling challenges related to fine-tuning and deployment.
- **Code Quality**: Clean, efficient, and well-documented.
- **API Design**: User-friendly documentation of REST API.
- **Deployment**: Scalable deployment ensuring reliability.

Task 2: Build an Intelligent Travel Planning AI Agent

Objective

Develop an AI Agent that helps users plan a **personalized 2-day trip** to any city (e.g., Tokyo, Paris, Singapore), using multiple tools, memory, and reasoning to simulate goal-driven, context-aware behavior.

Requirements

1. Tool Use & Decision-Making

Integrate at least **3 tools** (e.g., web search API, weather API, mock flight/hotel API, or attraction DB).
The agent must **decide** when and how to use tools dynamically.

2. Memory Management

Implement:

- **Short-term memory** (conversation context)
- **Long-term memory** using a **vector database** (e.g., Pinecone, FAISS) to remember user preferences across sessions.

3. Planning & Reasoning

- Use a **planning mechanism** (e.g., LangChain's Plan-and-Execute or custom logic) to break down user goals into sub-tasks.
- Include **multi-hop RAG** (retrieval-augmented generation) for detailed trip suggestions from external sources (e.g., Wikipedia dumps, blogs).

4. Conversational Interface

- Provide interaction via **chat API** (FastAPI preferred) or **simple web UI**.
- The agent should clarify ambiguous inputs and maintain context.

Deployment

- Deploy on a cloud platform (Render, AWS, etc.)

- Expose a **REST API** for interaction.

Deliverables

- Clean, modular codebase with setup guide
- REST API or UI demo
- Brief technical document (architecture, tool selection, memory design)
- (Optional) 2-minute demo video

Evaluation Criteria

Area	Focus
Agent Logic	Smart planning, reasoning, and tool selection
LLM + RAG Integration	Effective retrieval and response generation
Memory Use	Short- and long-term memory handling
Code Quality	Clean, modular, well-documented
Deployment	Working cloud deployment with API
Bonus	Constraints support (e.g., budget, pet-friendly), multilingual support

Task 3: Emotion Detection from Video using YOLO v8

Prompt:
Use YOLO v8 (or the latest version) to process real-time or recorded videos for detecting human facial emotions. This task combines object detection with emotion classification.

Requirements:

1. **Model Development**:
    - Fine-tune YOLO for face detection using a dataset like WIDER FACE.
    - Train an integrated emotion classification model using datasets like:
        - FER-2013
        - AffectNet
    - Classify emotions into categories: happy, sad, angry, surprised, neutral, etc.
2. **Pipeline**:
    - Preprocess videos to detect faces using YOLO.
    - Integrate YOLO with the emotion recognition module.
3. **Deployment**:
    - Deploy the pipeline on a GPU-enabled cloud platform.
    - Build a REST API to process video streams or files, returning emotions with bounding boxes for each frame.
4. **Real-Time Interface**:
    - Create a simple web app to display live results.

Evaluation Metrics:

- **Model Accuracy**: Precision, Recall, and F1-score on a test set.
- **Performance**: Real-time detection with a minimum FPS benchmark.

Evaluation Criteria:

- **Technical Skills**: Fine-tuning YOLO and integrating emotion recognition.
- **Problem-Solving**: Addressing challenges like lighting, occlusion, and frame rate optimization.
- **Code Quality**: Modular and efficient.
- **API Design**: Detailed API documentation.
- **Deployment**: Scalable and low-latency pipeline.